面试系列

1、初级面试题

由 涤生老师-健哥创建, 最后修改于一月 04, 2025

- 1、介绍下Kafka，Kafka的作用？适用场景？（了解）
- 2、Kafka作为消息队列，它可解决什么样的问题？（了解）
- 3、kafka核心架构及概念（必背）
- 4、auto.offset.reset参数的理解（必背）
- 5、Kafka 中的 ISR、AR 又代表什么？ISR 的伸缩又指什么（理解）
- 6、Kafka 中的 HW、LEO、LSO、LW 等分别代表什么？（理解）
- 7、四大主流MQ（kafka、ActiveMQ、RabbitMQ、RocketMQ）各自的优缺点（了解）
- 8、了解Kafka里的Controller吗？Controller的选举？职责？还有脑裂？（理解）
- 9、Kafka 中的分区器、序列化器、拦截器是否了解？它们之间的处理顺序是什么？（了解）
- 10、kafka的Partition分配策略有哪些（背诵）
- 11、Kafka 生产者客户端的整体结构是什么样子的？（理解）
- 12、Kafka 生产者客户端中使用了几个线程来处理？分别是什么？（理解）
- 13、Kafka 的旧版Scala 的消费者客户端的设计有什么缺陷？（了解）
- 14、“消费组中的消费者个数如果超过 topic 的分区，那么就会有消费者消费不到数据”这句话是否正确？如果正确，那么有没有什么 hack 的手段？（了解）
- 15、有哪些情形会造成重复消费？（理解）
- 16、哪些情景下会造成消息漏消费？（背诵）
- 17、KafkaConsumer是非线程安全的，那么怎么样实现多线程消费？（理解）
- 18、简述消费者与消费组之间的关系（理解）
- 19、当你使用kafka-topics.sh创建（删除）了一个topic之后，Kafka背后会执行什么逻辑？（了解）
- 20、topic的分区数可不可以增加？如果可以怎么增加？如果不可以，那又是为什么？（了解）
- 21、topic的分区数可不可以减少？如果可以怎么减少？如果不可以，那又是为什么？（理解）
- 22、创建topic时如何选择合适的分区数？（背诵）

1、介绍下Kafka，Kafka的作用？适用场景？（了解）

Kafka是一种分布式、高吞吐量的分布式发布订阅消息系统，它可以处理消费者规模的网站中的所有动作流数据，**主要应用于大数据实时处理领域**。简单地说，Kafka就相比是一个邮箱，生产者是发送邮件的人，消费者是接收邮件的人，Kafka就是用来存东西的，只不过它提供了一些处理邮件的机制。

i、作用

1）发布和订阅消息流

2）以容错的方式记录消息流，kafka以文件的方式来存储消息流

3）可以在消息发布的时候进行处理

ii、优势

高吞吐量、低延迟：kafka每秒可以处理几十万条消息，它的延迟最低只有几毫秒；

可扩展性：kafka集群支持热扩展；

持久性、可靠性：消息被持久化到本地磁盘，并且支持数据备份防止数据丢失；

容错性：允许集群中节点故障（若副本数量为n,则允许n-1个节点故障）；

高并发：支持数千个客户端同时读写。

iii、使用场景

消息系统： Kafka 和传统的消息系统（也称作消息中间件）都具备系统解耦、冗余存储、流量削峰、缓冲、异步通信、扩展性、可恢复性等功能。与此同时，Kafka 还提供了大多数消息系统难以实现的消息顺序性保障及回溯消费的功能。

存储系统： Kafka 把消息持久化到磁盘，相比于其他基于内存存储的系统而言，有效地降低了数据丢失的风险。也正是得益于 Kafka 的消息持久化功能和多副本机制，我们可以把Kafka 作为长期的数据存储系统来使用，只需要把对应的数据保留策略设置为“永久”或启用主题的日志压缩功能即可。

流式处理平台： Kafka 不仅为每个流行的流式处理框架提供了可靠的数据来源，还提供了一个完整的流式处理类库，比如窗口、连接、变换和聚合等各类操作。

2、Kafka作为消息队列，它可解决什么样的问题？（了解）

kafka是消息中间件的一种，它可以让合适的数据以合适的形式出现在合适的地方。Kafka的做法是提供消息队列，让生产者单往队列的末尾添加数据，让多个消费者从队列里面依次读取数据然后自行处理。Kafka主要用途是数据集成，或者说是流数据集成，以Pub/Sub形式的消息总线形式提供。但是，Kafka不仅仅是一套传统的消息总线，本质上Kafka是分布式的流数据平台，因为以下特性而著名：提供Pub/Sub方式的海量消息处理。以高容错的方式存储海量数据流。保证数据流的顺序。

缓冲和削峰：上游数据时有突发流量，下游可能扛不住，或者下游没有足够多的机器来保证冗余，kafka在中间可以起到一个缓冲的作用，把消息暂存在kafka中，下游服务就可以按照自己的节奏进行慢慢处理。

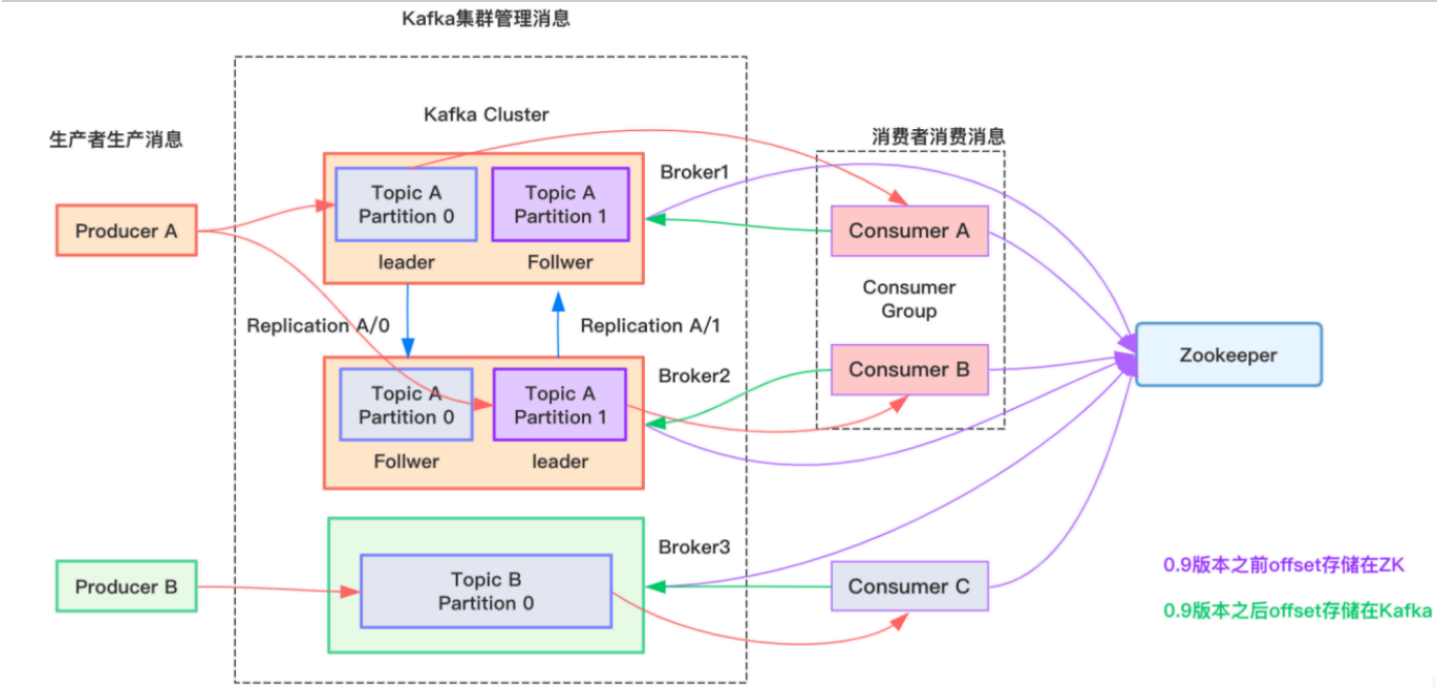
解耦和扩展性：项目开始的时候，并不能确定具体需求。消息队列可以作为一个接口层，解耦重要的业务流程。只需要遵守约定，针对数据编程即可获取扩展能力。

冗余：可以采用一对多的方式，一个生产者发布消息，可以被多个订阅topic的服务消费到，供多个毫无关联的业务使用。

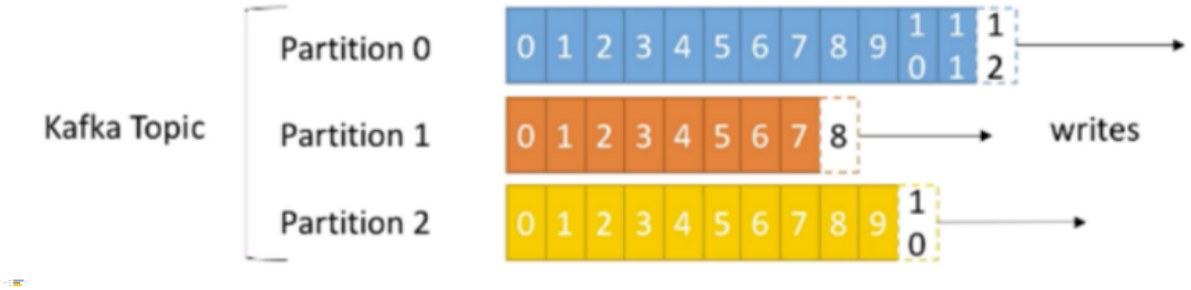
健壮性：消息队列可以堆积请求，所以消费端业务即使短时间死掉，也不会影响主要业务的正常进行。

异步通信：很多时候，用户不想也不需要立即处理消息。消息队列提供了异步处理机制，允许用户把一个消息放入队列，但并不立即处理它。想向队列中放入多少消息就放多少，然后在需要的时候再去处理它们。

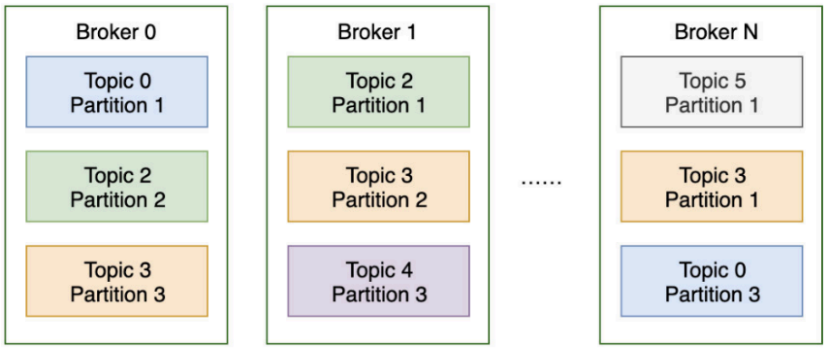
面试系列



- Kafka 集群包含一个或多个服务器，服务器节点称为**Broker，是物理概念**；
- **Producer** 就是负责向Kafka 服务端写入数据的程序；
- **Topic是一个逻辑概念**，可以看做是一个消息类别的名称，同类消息发送到同一个Topic下面，比如订单消息，就只发送到订单Topic下面，即 **kafka 是面向 topic 的；**物理上不同的topic对应的消息是分开存储的。topic就好比是数据库的表，尤其是分库分表之后的逻辑表。
- **Partition 是物理上的概念**，每个 topic 包含一个或多个 partition，因此partition 是最小的存储单元，存储着一个 topic 的部分数据。**每个 partition 都是一个单独的 log 文件，每条记录都以追加的形式写入**。当生产者产生数据的时候，根据分配策略，选择分区，然后将消息追加到指定的分区的末尾（队列），而消费者消费的时候又是从头开始消费的，这也是Kafka为什么被称为MQ的原因所在。



为了做到均匀分布，通常partition的数量通常是Broker Server数量的整数倍。因此，从这里我们可以看出Broker、Topic和Partition之间的关系了。



- **Replica**是partition 的副本，保障 partition 的高可用。副本集中主要有两个角色：

  i、leader：replica 中的一个角色， producer 和 consumer 只跟 leader 交互。

  ii、follower：replica 中的一个角色，从 leader 中复制数据。

  总结：每个partition有多个副本，其中有且仅有一个作为Leader，Leader是当前负责数据的读写的partition。

  replica 的分配有**三个目标**：

  i、在brokers之间**均分**replicas

  ii、partition与它的其他replicas**不再**同一个broker上

  iii、如果broker有rack信息，则partition的replicas尽量分配在不同rack上面

- **Consumer**是从 kafka 集群中消费消息的终端或服务；
- **每个Consumer都属于一个Consumer Group，是逻辑上的概念，同一个topic里的数据，可以由不同的consumergroup里的consumer进行消费**；但是**同一个group里的不同consumer只能消费不同的partition。** consumer的数量通常不超过partition的数量，且二者最好保持整数倍关系，因为**Kafka在设计时假定了一个partition只能被同一个group内的一个consumer消费。**如果consumer group中的consumer线程数量比partition少，那么有的线程将会收到多个消息。由于kafka只能保证在一个partition上数据是有序的，那么如果一个consumer消费一个topic的多个分区，那么就会出现数据乱序；

因此理解consumer group记住下面这**三个特性**就好了：

中，一定要给group.id设置一个业务唯一的名字，防止业务错乱

iii、consumer group下订阅的topic下的每个分区只能分配给某个group下的一个consumer(当然该分区还可以被分配给其他group)

- 消费者在消费的过程中需要记录自己消费了多少数据，即消费位置信息，在Kafka中这个位置信息有个专门的术语：**位移(offset)**。offset表明了某个 Consumer Group 在不同 Topic Partition 上面的消费偏移量（也可以理解为消费进度），它记录了 Consumer 要消费的下一条消息的位移。
- **Kafka把它的meta数据都存储在Zookeeper上，所以说ZK是目前kafka的重要的依赖**，没有ZK没法运行Kafka；在老版本（0.8.1以前）里面消费段（consumer）也是依赖ZK的，在新版本中移除了客户端对ZK的依赖，但是broker依然依赖于ZK。

4、auto.offset.reset参数的理解（必背）

可以通过参数**auto.offset.reset**指定 Offset 消费，主要有以下几个取值：

earliest：当各分区下有已提交的offset时，从最近一次提交的offset开始消费；无提交的offset时，从头开始消费

latest：当各分区下有已提交的offset时，从最近一次提交的offset开始消费；无提交的offset时，消费新产生的该分区下的数据，**默认值**

none：当各分区都存在已提交的offset时，从最近一次提交的offset后开始消费；只要有一个分区不存在已提交的offset，则抛出异常

对于已经提交过offset，none、latest和earliest没有区别，但是在没有提交offset情况下，用latest直接会导致无法读取旧数据。因此只要不更改group.id，每次重新消费kafka，都是从上次消费结束的地方继续开始，不论"auto.offset.reset"属性设置的是什么

5、Kafka 中的 ISR、AR 又代表什么？ISR 的伸缩又指什么（理解）

分区中的所有副本统称为 AR（Assigned Replicas）。所有与 leader 副本保持一定程度同步的副本（包括 leader 副本在内）组成 **ISR（In-Sync Replicas）**，ISR 集合是 AR 集合中的一个子集。Follower 与 leader 副本同步时，延迟过多的副本放在**OSR（Out Sync Replicas）集合中**。因此可以得到一个结论是：AR = ISR + OSR

ISR 的伸缩：leader 副本负责维护和跟踪 ISR 集合中所有 follower 副本的滞后状态，当 follower 副本落后太多或失效时，leader 副本会把它从 ISR 集合中剔除。如果 OSR 集合中有follower 副本"追上"了 leader 副本，那么 leader 副本会把它从 OSR 集合转移至 ISR集合。默认情况下，当 leader 副本发生故障时，只有在 ISR 集合中的副本才有资格被选举为新的 leader，而在 OSR 集合中的副本则没有任何机会（不过这个原则也可以通过修改相应的参数配置来改变）。

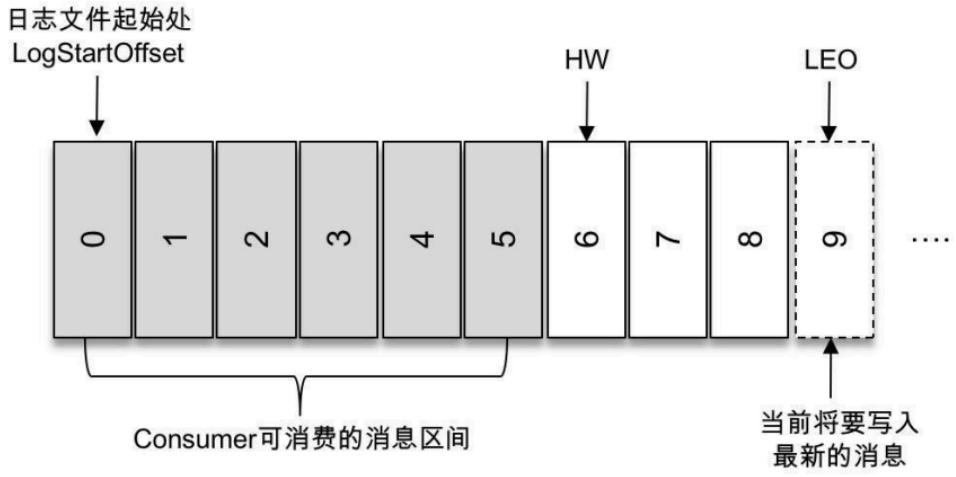
replica.lag.time.max.ms： 这个参数的含义是 Follower 副本能够落后 Leader 副本的最长时间间隔，当前默认值是 10 秒。

unclean.leader.election.enable：是否允许 Unclean 领导者选举。开启 Unclean 领导者选举可能会造成数据丢失，但好处是，它使得分区 Leader 副本一直存在，不至于停止对外提供服务，因此提升了高可用性。

6、Kafka 中的 HW、LEO、LSO、LW 等分别代表什么？（理解）

HW 是 High Watermark 的缩写，俗称高水位，它标识了一个特定的**消息消费偏移量offset，消费者只能拉取到这个 offset 之前的消息。**

LSO 是LogStartOffset，一般情况下，日志文件的起始偏移量 logStartOffset 等于第一个日志分段的 baseOffset，但这并不是绝对的，logStartOffset 的值可以通过 DeleteRecordsRequest 请求(比如使用 KafkaAdminClient 的deleteRecords()方法、使用kafka-delete-records.sh 脚本、日志的清理和截断等操作进行修改。



如上图所示，它代表一个日志文件，这个日志文件中有 9 条消息，**第一条消息的 offset（LogStartOffset）为 0**，最后一条消息的 offset 为 8，offset 为 9 的消息用虚线框表示，代表下一条待写入的消息。日志文件的 HW 为 6，表示消费者只能拉取到 offset 在 0 至5之间的消息，而 **offset 为 6 的消息对消费者而言是不可见的。**

LEO 是 Log End Offset 的缩写，它标识当前日志文件中下一条待写入消息的 offset，是**生产偏移量offset**，上图中 offset 为 9 的位置即为当前日志文件的 LEO，**LEO 的大小相当于当前日志分区中最后一条消息的 offset 值加 1**。分区 ISR 集合中的每个副本都会维护自身的 LEO，而 ISR 集合中最小的 LEO 即为分区的 HW，对消费者而言只能消费 HW 之前的消息。

LW 是 Low Watermark 的缩写，俗称"低水位"，代表 AR 集合中最小的 logStartOffset 值。副本的拉取请求(FetchRequest，它有可能触发新建日志分段而旧的被清理，进而导致 logStartOffset 的增加)和删除消息请求(DeleteRecordRequest)都有可能促使 LW 的增长。

面试系列

–	RabbitMQ	ActiveMQ	RocketMQ	Kafka
所属社区/公司	Mozilla Public License	Apache	Ali	Apache
成熟度	成熟	成熟	比较成熟	成熟
授权方式	开源	开源	开源	开源
开发语言	Erlang	Java	Java	Scala&Java
客户端支持语言	官方支持Erlang, Java, Ruby等, 社区产出多种语言API， 几乎支持所有常用语言	Java、C、C++、Python、PHP、Perl、.net 等	Java C++（不成熟）	官方支持Java, 开源社区有多语言版本, 如PHP, Python, Go, C/C++, Ruby, NodeJS等编程语言, 详见Kafka 客户端列表
协议支持	多协议支持:AMQP，XMPP, SMTP, STOMP	OpenWire、STOMP、REST、XMPP、AMQP	自己定义的一套(社区提供JMS--不成熟)	自有协议，社区封装了HTTP协议支持
消息批量操作	不支持	支持	支持	支持
消息推拉模式	多协议, Pull/Push均有支持	多协议, Pull/Push均有支持	多协议, Pull/Push均有支持	Pull
HA	master/slave模式, master提供服务, slave仅作备份	基于ZooKeeper + LevelDB 的 Master-Slave 实现方式	支持多Master 模式、多 Master 多 Slave 模式，异步复制模式、多 Master 多 Slave 模式，同步双写	支持replica机制, leader宕掉后, 备份自动顶替， 并重新选举leader(基于Zookeeper)
数据可靠性	可以保证数据不丢, 有slave用作备份	master/slave	支持异步实时刷盘，同步刷盘，同步复制，异步复制	数据可靠， 并且有replica机制, 有容错容灾能力
单机吞吐量	其次(万级)	最差(万级)	最高(十万级)	次之(十万级)
消息延迟	微秒级	\	比kafka快	毫秒级
持久化能力	内存、文件，支持数据堆积,但数据堆积反过来影响生产速率	内存、文件、数据库	磁盘文件	磁盘文件, 只要磁盘容量够， 可以做到无限消息堆积
是否有序	若想有序，只能使用一个Client	可以支持有序	有序	多Client保证有序
事务	不支持	支持	支持	不支持, 但可以通过Low Level API保证仅消费一次
集群	支持	支持	支持	支持
负载均衡	支持	支持	支持	支持
管理界面	较好	一般	命令行界面	官方只提供了命令行版, Yahoo开源自己的Kafka Web管理界面Kafka-Manager
部署方式	独立	独立	独立	独立

8、了解Kafka里的Controller吗？Controller的选举？职责？还有脑裂？（理解）

一般集群模式分为主从模式、主备模式等。但是在kafka集群中各broker间不存在主从、主备的关系。 但是**为了能够更好的管理broker上下线、topic分区等工作，会在broker之间会选取出Controller的角色，由当前角色做管理工作。** 整个集群中只会出现一个Controller角色。Controller一般是与Zookeeper协助管理整个Kafka集群。

既然Controller是由broker间的选举，那么它们的选举流程是什么呢？ 其实非常的简单：**在每个broker启动时，都会与zookeeper进行交互，而第一个在zookeeper上成功创建/controller目录的broker节点即被选举为Controller**

当某个broker成功当选Controller之后，那么它将为整个Kafka集群服务，包括但不限定于：

- **i、集群内Broker节点管理： 包括新增、下线等**
- **ii、副本Leader选举、分区重分配**

总之，**所有与Zookeeper相交互的工作，都是Controller角色来完成**

由于集群属于分布式环境，进而有多种因素会造成Broker与Zookeeper间的会话失效，例如：网络波动造成Controller与Zookeeper连接会话失效、长时间GC无响应导致与Zookeeper连接会话失效等。而如上原因也必将**导致旧Controller无法接收到新Controller选举结果，那么当网络或者GC恢复之后，由于缺少了部分记忆，旧Controller还以为自己是Controller角色，在集群中就出现两个Controller角色，发出具有冲突性的命令。从而导致脑裂现象出现。**

而脑裂问题不解决，会导致严重数据不一致现象。故而在Kafka中引入了**epoch number**来解决这个问题。epoch number仅仅是一个**单调递增**的数字，每次Controller角色的选举都会加1，同时Zookeeper回调会将当前epoch number通知到其它broker节点。**如果在Controller发布的命令中，包含了比当前epoch number更小的值，那么broker会忽略这条命令，不做任何操作。**

9、Kafka 中的分区器、序列化器、拦截器是否了解？它们之间的处理顺序是什么？（了解）

序列化器：生产者需要用序列化器（Serializer）把对象转换成字节数组才能通过网络发送给 Kafka。而在对侧，消费者需要用反序列化器（Deserializer）把从 Kafka中收到的字节数组转换成相应的对象。

分区器：分区器的作用就是为消息分配分区。如果消息 ProducerRecord 中没有指定 partition 字段，那么就需要依赖分区器，根据 key 这个字段来计算 partition的值。

如统计类工作。

消费者拦截器主要在消费到消息或在提交消费位移时进行一些定制化的操作。

消息在通过 send() 方法发往 broker 的过程中，有可能需要经过拦截器（Interceptor）、序列化器（Serializer）和分区器（Partitioner）的一系列作用之后才能被真正地发往 broker。拦截器一般不是必需的，而序列化器是必需的。消息经过序列化之后就需要确定它发往的分区，如果消息 ProducerRecord 中指定了 partition 字段，那么就不需要分区器的作用，因为 partition 代表的就是所要发往的分区号。

处理顺序： 拦截器->序列化器->分区器

KafkaProducer 在将消息序列化和计算分区之前会调用生产者拦截器的 onSend() 方法来对消息进行相应的定制化操作。然后生产者需要用序列化器（Serializer）把对象转换成字节数组才能通过网络发送给Kafka。最后可能会被发往分区器为消息分配分区。

10、kafka的Partition分配策略有哪些（背诵）

在实际使用场景中，消费者组内的每个消费者消费了哪些分区的数据呢？**理想情况下，Group 内的 Consumer 实例的数量应该等于该 Group 订阅 topic 的分区总数。因此 Partition 的分配问题: 确定哪个 Partition 由哪个 Consumer 来消费的问题。**

每个消费者客户端都可以通过 **partition.assignment.strategy** 属性配置分区分配策略，并且**可以同时配置多个分区分配策略（用列表表示）**。不同的 Kafka 版本的默认分区策略可能不同，比如在 Kafka 2.0 ~ 2.8 版本中，默认的分区策略是 RangeAssignor，在 Kafka 3.0 以后，partition.assignment.strategy 的默认分区策略是 [RangeAssignor, CooperativeStickyAssignor]，**表达的含义是：当消费者组启动以后默认使用 RangeAssignor 策略，但是当触发一次 Rebalance 之后，就会移除 RangeAssignor 协议，转而使用 CooperativeStickyAssignor**。

i、**RangeAssignor 是 Kafka 默认的分区分配算法**，它是按照 Topic 的维度进行分配的，对于每个 Topic，**首先对 Partition 按照分区ID进行排序，然后对订阅这个 Topic 的 Consumer Group 的 Consumer 再进行排序，之后尽量均衡的按照范围区段将分区分配给 Consumer，**此时可能会造成先分配分区的 Consumer 进程的任务过重（分区数无法被消费者数量整除）。不客气的说，这种策略毫无全局观念。

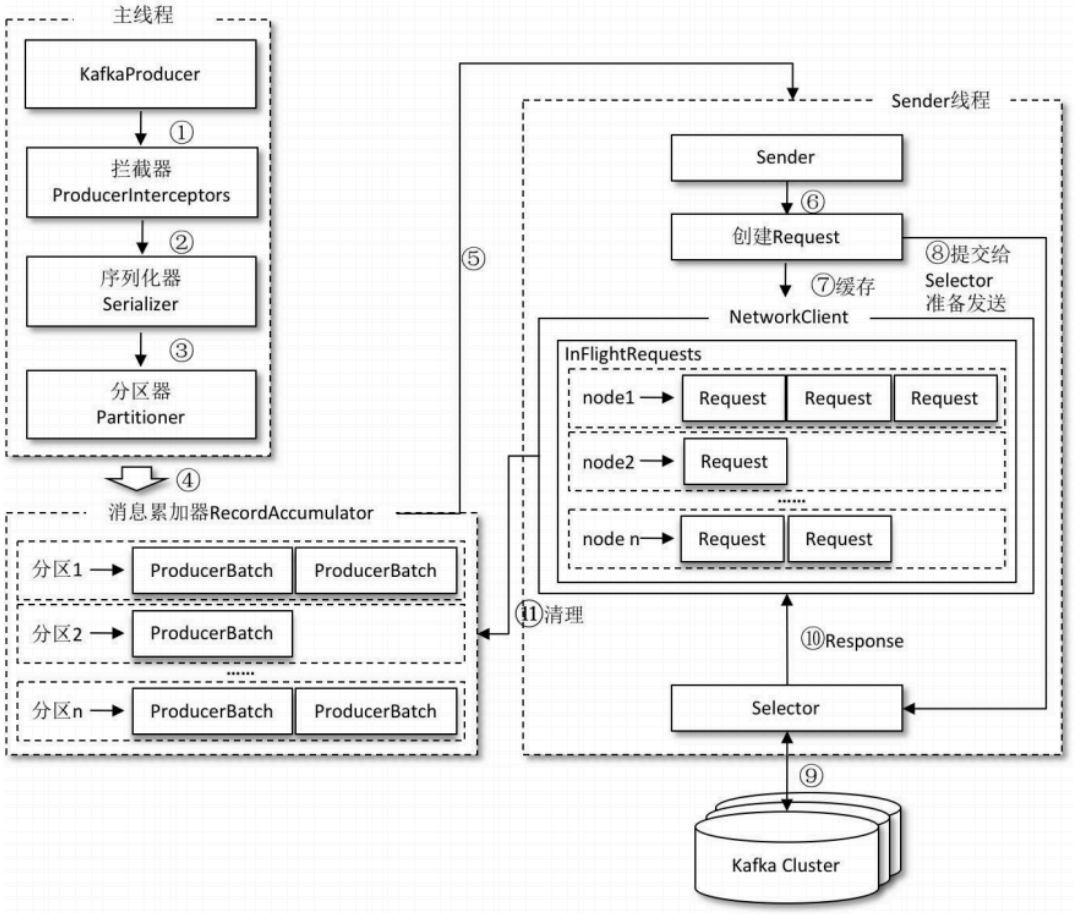
ii、**RoundRobinAssignor** 的分区分配策略是**将 Consumer Group 内订阅的所有 Topic 的 Partition 及所有 Consumer 进行排序后按照顺序尽量均衡的一个一个进行分配（即轮询分配）**。如果 Consumer Group 内，每个 Consumer 都订阅了相同的Topic，那么分配结果是均衡的。如果订阅 Topic 是不同的，那么分配结果是不保证“尽量均衡”的，因为某些 Consumer 可能不参与一些 Topic 的分配。

iii、上面介绍的两种分区策略都或多或少存在分配不均衡的问题，最主要的问题是在发生 Rebalance 时会将所有的分区再重新计算并分配一遍，每次分配的结果会有比较大的偏差。**StickyAssignor 分区策略是 RoundRobinAssignor 的变种，它们的分配逻辑大致相同，但在 Rebalance 时会尽最大努力保持分区方案的平衡，这样做可以快速完成重新平衡操作，效率更高。** StickyAssignor 分区分配算法是 Kafka Java 客户端提供的分配策略中最复杂的一种，从 0.11 版本开始引入，目的就是在执行新分配时，**尽量在上一次分配结果上少做调整**，其主要实现了以下2个目标：

1)Topic Partition 的分配要尽量均衡；

2)当 Rebalance(重分配，后面会详细分析) 发生时，尽量与上一次分配结果保持一致。

11、Kafka 生产者客户端的整体结构是什么样子的？（理解）



整个生产者客户端由两个线程协调运行，这两个线程分别为**主线程和 Sender 线程（发送线程）**。在主线程中由 KafkaProducer 创建消息，然后通过可能的拦截器、序列化器和分区器的作用之后缓存到消息累加器（RecordAccumulator，也称为消息收集器）中。Sender 线程负责从 RecordAccumulator 中获取消息并将其发送到 Kafka 中。**RecordAccumulator 主要用来缓存消息以便 Sender 线程可以批量发送，进而减少网络传输的资源消耗以提升性能。**

面试系列

整个生产者客户端由两个线程协调运行，这两个线程分别为主线程和 Sender 线程（发送线程）。在主线程中由 KafkaProducer 创建消息，然后通过可能的拦截器、序列化器和分区器的作用之后缓存到消息累加器（RecordAccumulator，也称为消息收集器）中。Sender 线程负责从 RecordAccumulator 中获取消息并将其发送到 Kafka 中。

13、Kafka 的旧版Scala 的消费者客户端的设计有什么缺陷？（了解）

老版本的 Consumer Group 把位移保存在 ZooKeeper 中。Apache ZooKeeper 是一个分布式的协调服务框架，Kafka 重度依赖它实现各种各样的协调管理。将位移保存在 ZooKeeper 外部系统的做法，最显而易见的好处就是减少了 Kafka Broker 端的状态保存开销。**ZooKeeper 这类元框架其实并不适合进行频繁的写更新，而 Consumer Group 的位移更新却是一个非常频繁的操作。这种大吞吐量的写操作会极大地拖慢 ZooKeeper 集群的性能**

14、“消费组中的消费者个数如果超过 topic 的分区，那么就会有消费者消费不到数据”这句话是否正确？如果正确，那么有没有什么 hack 的手段？（了解）

一般来说如果消费者过多，出现了消费者的个数大于分区个数的情况，就会有消费者分配不到任何分区。开发者可以继承 AbstractPartitionAssignor 实现自定义消费策略，从而实现同一消费组内的任意消费者都可以消费订阅主题的所有分区：

```
public class BroadcastAssignor extends AbstractPartitionAssignor{
    @Override
    public String name() {
        return "broadcast";
    }

    private Map<String, List<String>> consumersPerTopic(Map<String, Subscription> consumerMetadata) {
        （具体实现请参考 RandomAssignor 中的 consumersPerTopic()方法）
    }

    @Override
    public Map<String, List<TopicPartition>> assign(Map<String, Integer> partitionsPerTopic,Map<String, Subscription> subscriptions)
        Map<String, List<String>> consumersPerTopic = consumersPerTopic(subscriptions);
        Map<String, List<TopicPartition>> assignment = new HashMap<>();
        //Java8
        subscriptions.keySet().forEach(memberId ->assignment.put(memberId, new ArrayList<>()));
        //针对每一个主题，为每一个订阅的消费者分配所有的分区
        consumersPerTopic.entrySet().forEach(topicEntry->{
            String topic = topicEntry.getKey();
            List<String> members = topicEntry.getValue();
            Integer numPartitionsForTopic = partitionsPerTopic.get(topic);
            if (numPartitionsForTopic == null || members.isEmpty())
                return;
            List<TopicPartition> partitions = AbstractPartitionAssignor.partitions(topic, numPartitionsForTopic);
            if (!partitions.isEmpty()) {
                members.forEach(memberId -> assignment.get(memberId).addAll(partitions));
            }
        });
    return assignment;
    }
}
```

注意组内广播的这种实现方式会有一个严重的问题—默认的消费位移的提交会失效。

15、有哪些情形会造成重复消费？（理解）

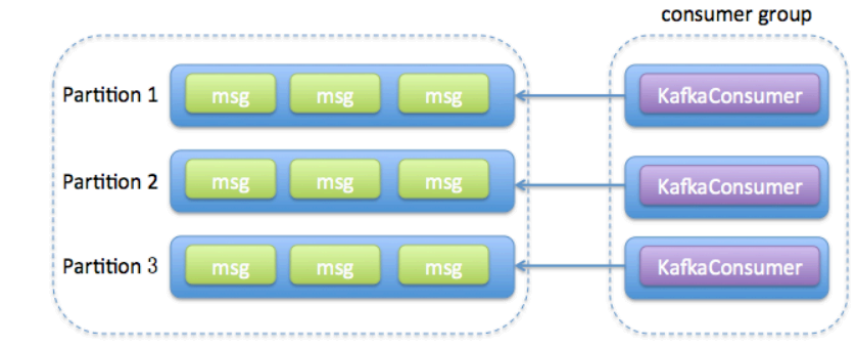
1. Rebalance
   一个consumer正在消费一个分区的一条消息，还没有消费完，发生了rebalance(加入了一个consumer)，从而导致这条消息没有消费成功，rebalance后，另一个consumer又把这条消息消费一遍。
2. 消费者端手动提交
   如果先消费消息，再更新offset位置，导致消息重复消费。
3. 消费者端自动提交
   设置offset为自动提交，关闭kafka时，如果在close之前，调用 consumer.unsubscribe() 则有可能部分offset没提交，下次重启会重复消费。
4. 生产者端
   生产者因为业务问题导致的宕机，在重启之后可能数据会重发

16、哪些情景下会造成消息漏消费？（背诵）

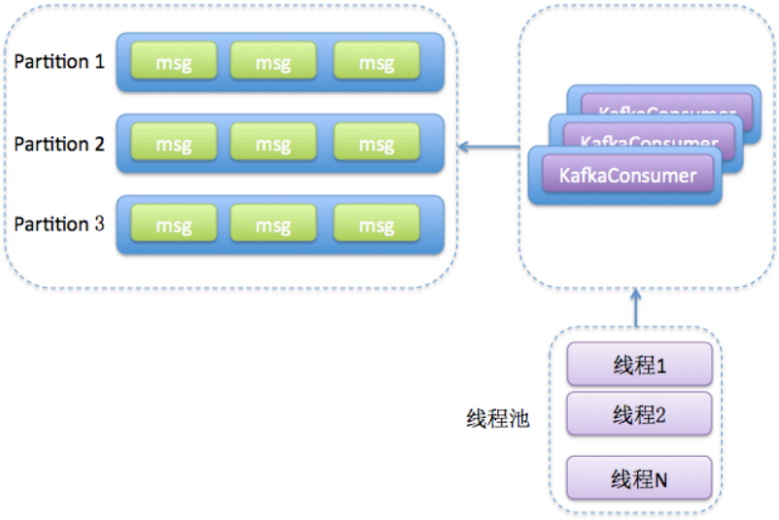
1. 自动提交
   设置offset为自动定时提交，当offset被自动定时提交时，数据还在内存中未处理，此时刚好把线程kill掉，那么offset已经提交，但是数据未处理，导致这部分内存中的数据丢失。
2. 生产者发送消息
   发送消息设置的是fire-and-forget（发后即忘），它只管往 Kafka 中发送消息而并不关心消息是否正确到达。不过在某些时候（比如发生不可重试异常时）会造成消息的丢失。这种发送方式的性能最高，可靠性也最差。
3. 消费者端
   先提交位移，但是消息还没消费完就宕机了，造成了消息没有被消费。自动位移提交同理
4. acks没有设置为all
   如果在broker还没把消息同步到其他broker的时候宕机了，那么消息将会丢失

面试系列

1、线程封闭，即为每个线程实例化一个 KafkaConsumer 对象



一个线程对应一个 KafkaConsumer 实例，我们可以称之为消费线程。一个消费线程可以消费一个或多个分区中的消息，所有的消费线程都隶属于同一个消费组。2、消费者程序使用单或多线程获取消息，同时创建多个消费线程执行消息处理逻辑。获取消息的线程可以是一个，也可以是多个，每个线程维护专属的KafkaConsumer 实例，处理消息则交由特定的线程池来做，从而实现消息获取与消息处理的真正解耦。具体架构如下图所示：



当然，这种方法还可以有多个变种：比如每个worker线程有自己的处理队列。consumer根据某种规则或逻辑将消息放入不同的队列。不过总体思想还是相同的，故这里不做过多展开讨论了。
下表总结了两种方法的优缺点：

	优点	缺点
方法1(每个线程维护一个KafkaConsumer)	方便实现 速度较快，因为不需要任何线程间交互 易于维护分区内的消息顺序	更多的TCP连接开销(每个线程都要维护若干个TCP连接) consumer数受限于topic分区数，扩展性差 频繁请求导致吞吐量下降 线程自己处理消费到的消息可能会导致超时，从而造成rebalance
方法2 (单个(或多个)consumer，多个worker线程)	可独立扩展consumer数和worker数，伸缩性好	实现麻烦 通常难于维护分区内的消息顺序 处理链路变长，导致难以保证提交位移的语义正确性

18、简述消费者与消费组之间的关系（理解）

1. Consumer Group 下可以有一个或多个 Consumer 实例。这里的实例可以是一个单独的进程，也可以是同一进程下的线程。在实际场景中，使用进程更为常见一些。
2. Group ID 是一个字符串，在一个 Kafka 集群中，它标识唯一的一个 Consumer Group。
3. Consumer Group 下所有实例订阅的主题的单个分区，只能分配给组内的某个 Consumer 实例消费。这个分区当然也可以被其他的 Group 消费。

19、当你使用kafka-topics.sh创建（删除）了一个topic之后，Kafka背后会执行什么逻辑？（了解）

在执行完脚本之后，Kafka 会在 log.dir 或 log.dirs 参数所配置的目录下创建相应的主题分区，默认情况下这个目录为/tmp/kafka-logs/。

在 ZooKeeper 的/brokers/topics/目录下创建一个同名的实节点，该节点中记录了该主题的分区副本分配方案。示例如下：

```
[zk: localhost:2181/kafka(CONNECTED) 2] get /brokers/topics/topic-create {"version":1,"partitions":{"2":[1,2],"1":[0,1],"3":[2,1],"0":[2,0]}}
```

20、topic的分区数可不可以增加？如果可以怎么增加？如果不可以，那又是为什么？（了解）

可以增加，使用 kafka-topics 脚本，结合 --alter 参数来增加某个主题的分区数，命令如下：

```
bin/kafka-topics.sh --bootstrap-server broker_host:port --alter --topic <topic_name> --partitions <新分区数>
```

当分区数增加时，就会触发订阅该主题的所有 Group 开启 Rebalance。
首先，Rebalance 过程对 Consumer Group 消费过程有极大的影响。在 Rebalance 过程中，所有 Consumer 实例都会停止消费，等待 Rebalance 完成。这是 Rebalance 为人诟病的一个方面。
其次，目前 Rebalance 的设计是所有 Consumer 实例共同参与，全部重新分配所有分区。其实更高效的做法是尽量减少分配方案的变动。
最后，Rebalance 实在是太慢了。

面试系列

不支持，因为删除的分区中的消息不好处理。如果直接存储到现有分区的尾部，消息的时间戳就不会递增，如此对于 Spark、Flink 这类需要消息时间戳（事件时间）的组件将会受到影响；如果分散插入现有的分区，那么在消息量很大的时候，内部的数据复制会占用很大的资源，而且在复制期间，此主题的可用性又如何得到保障？与此同时，顺序性问题、事务性问题，以及分区和副本的状态机切换问题都是不得不面对的。

22、创建topic时如何选择合适的分区数？（背诵）

在 Kafka 中，性能与分区数有着必然的关系，在设定分区数时一般也需要考虑性能的因素。对不同的硬件而言，其对应的性能也会不太一样。可以使用Kafka 本身提供的用于生产者性能测试的 kafka-producer- perf-test.sh 和用于消费者性能测试的 kafka-consumer-perf-test.sh来进行测试。

增加合适的分区数可以在一定程度上提升整体吞吐量，但超过对应的阈值之后吞吐量不升反降。如果应用对吞吐量有一定程度上的要求，则建议在投入生产环境之前对同款硬件资源做一个完备的吞吐量相关的测试，以找到合适的分区数阈值区间。

分区数的多少还会影响系统的可用性。如果分区数非常多，如果集群中的某个 broker 节点宕机，那么就会有大量的分区需要同时进行 leader 角色切换，这个切换的过程会耗费一笔可观的时间，并且在这个时间窗口内这些分区也会变得不可用。

分区数越多也会让 Kafka 的正常启动和关闭的耗时变得越长，与此同时，主题的分区数越多不仅会增加日志清理的耗时，而且在被删除时也会耗费更多的时间。

郑冬综合2502 2026/5/26 22:05:16
涤生大数据版权所有，请勿外传

无标签